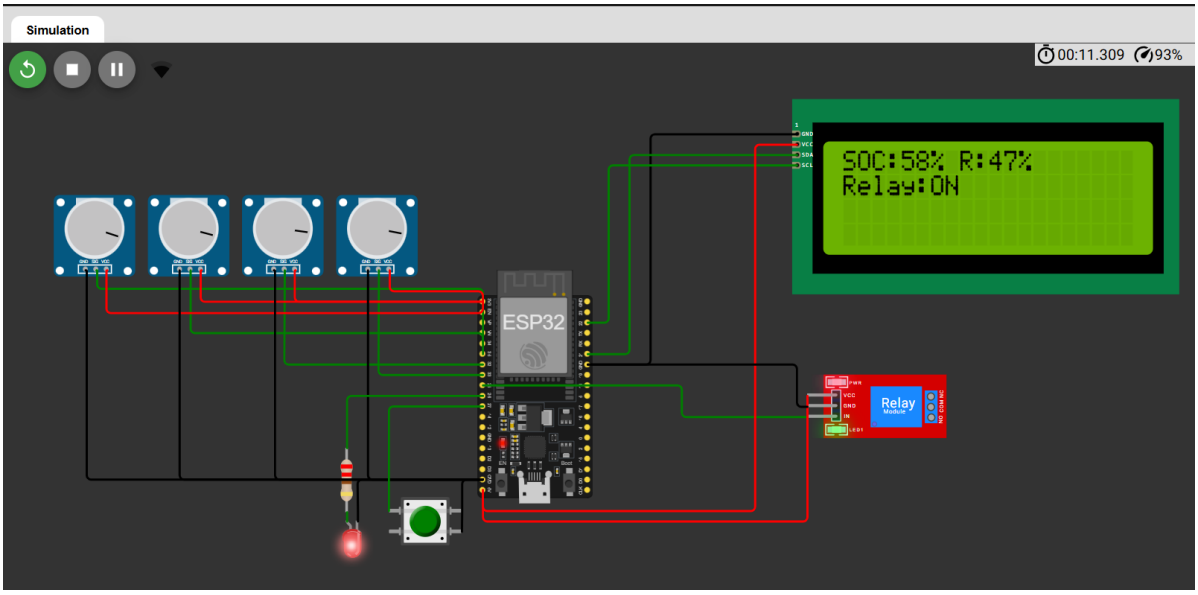
PROJECT REPORT

ESP32-Based Intelligent Battery Management System

A simulation-based prototype for cell monitoring, fault protection, LCD status display, Wi-Fi connectivity and Blynk IoT telemetry



Submitted by
Parth Nandalal Khairnar
Electronics and Telecommunication Engineering
Dr. D. Y. Patil College of Engineering, Akurdi, Pune

Project documentation | 2026

Abstract

This project presents a simulation-based intelligent Battery Management System (BMS) built around an ESP32 development board. Four potentiometers represent four battery-cell voltage inputs. The firmware samples and analyses the cell readings, identifies the weakest and strongest cells, calculates voltage imbalance, estimates state of charge (SoC), evaluates a demonstration risk score, and manages protection states. A 16×2 I²C LCD provides local status information, while a relay output and LED indicate protection actions. Wi-Fi and Blynk provide remote monitoring and historical voltage visualisation. The prototype was developed and demonstrated in Wokwi; its results validate the intended control flow and interface behaviour in simulation, not the safety performance of a real battery pack.

Keywords

ESP32, Battery Management System, cell voltage monitoring, voltage imbalance, relay protection, fault-state machine, LCD, Wi-Fi, Blynk IoT, Wokwi.

Table of Contents

- 1. Introduction
- 2. Problem Statement and Objectives
- 3. System Architecture
- 4. Hardware and Software Requirements
- 5. Circuit Connections
- 6. Firmware Design
- 7. Monitoring and Analytics
- 8. Testing and Results
- 9. Screenshots and Evidence
- 10. Limitations
- 11. Future Scope
- 12. Conclusion
- Appendix A: Pin Map
- Appendix B: Repository and Reproducibility

1. Introduction

A Battery Management System monitors battery conditions and helps protect cells from operating outside defined limits. Cell-to-cell voltage differences can indicate imbalance, while undervoltage or abnormal readings may require protective action. This project demonstrates the core monitoring and supervisory logic using an ESP32 and a Wokwi simulation. The use of potentiometers allows each cell input to be varied independently so that normal, imbalanced and fault conditions can be tested without a physical battery pack.

The system combines local display, relay/LED outputs, state management, Wi-Fi connectivity and a Blynk dashboard. The design is intended as an educational prototype for embedded systems, IoT telemetry and fault-handling workflows.

2. Problem Statement and Objectives

2.1 Problem statement

Battery packs contain multiple cells whose voltages may differ during use. A monitoring system should make these differences visible, identify abnormal cell readings, and indicate when protective action is required. It should also present essential information locally and remotely, while handling communication interruptions without blocking the main monitoring loop.

2.2 Objectives

- Read and display four independently adjustable cell-voltage inputs.
- Identify the weakest and strongest cells and calculate pack voltage imbalance.
- Track imbalance trend and compare the measured imbalance with a configured threshold.
- Demonstrate fault states including NORMAL, DEGRADED, FAILSAFE and SHUTDOWN.
- Control a relay output and indicator LED according to protection logic.
- Refresh LCD pages without using a blocking delay in the main loop.
- Send telemetry to Blynk and display current values and historical voltage graphs.
- Provide demonstration analytics such as SoC estimate, risk score and maintenance advice.

3. System Architecture

The prototype follows a layered structure. The sensing layer reads four analog inputs. The analysis layer filters readings, finds minimum and maximum cell voltages, calculates imbalance and estimates SoC. The protection layer evaluates faults and updates the system state and relay output. The presentation layer updates the LCD and serial monitor. The connectivity layer maintains Wi-Fi/Blynk status and sends telemetry. A dashboard layer displays current values and historical cell-voltage data.

Layer	Role	Implementation in prototype
Sensing	Acquire cell inputs	Four potentiometers connected to ESP32 ADC pins
Analysis	Calculate battery indicators	Weakest/strongest cell, imbalance, trend, estimated SoC and risk score
Protection	Handle abnormal conditions	State machine, fault identifier, relay and LED output

Local UI	Show immediate status	16×2 I²C LCD with rotating pages and fault display
Connectivity	Remote telemetry	Wi-Fi and Blynk connection status
Dashboard	Visualise measurements	Blynk value widgets and four historical voltage charts

4. Hardware and Software Requirements

4.1 Hardware components

- ESP32 DevKit V1 development board.
- Four potentiometers used as simulated cell-voltage inputs.
- 16×2 LCD with I²C interface.
- Single-channel relay module for demonstrating a protection output.
- LED and series current-limiting resistor.
- Push button for manual state interaction.
- Wokwi simulator for wiring and behavioural tests.

4.2 Software and services

- Wokwi — circuit simulation and firmware testing.
- Arduino-style ESP32 firmware.
- LiquidCrystal_I2C library — LCD control.
- Blynk IoT — live dashboard and historical datastream graphs.
- GitHub — source-code versioning and project documentation.

5. Circuit Connections

The following pin map reflects the project configuration visible in the source and simulation screenshots. Verify the actual diagram before moving from simulation to physical hardware.

Component / signal	ESP32 pin / setting	Purpose
Cell 1 potentiometer output	GPIO34	Analog cell 1 input
Cell 2 potentiometer output	GPIO35	Analog cell 2 input
Cell 3 potentiometer output	GPIO32	Analog cell 3 input
Cell 4 potentiometer output	GPIO33	Analog cell 4 input
LCD SDA	GPIO21	I²C data
LCD SCL	GPIO22	I²C clock
Relay module IN	GPIO25	Protection output
Indicator LED	GPIO26	Visual indication through resistor
Push button	GPIO27	Button input using INPUT_PULLUP

LCD I²C address	0x27	LCD device address used by sketch

All potentiometers share the simulator's supply and ground references. Each wiper is routed to its corresponding ADC pin. In real hardware, never connect a battery cell directly to an ESP32 ADC pin unless a properly designed and rated measurement/protection front end is used.

6. Firmware Design

6.1 Periodic, non-blocking loop

The main loop services Blynk and uses elapsed-time checks based on millis() to schedule sensor acquisition, analytics, LCD rotation and telemetry. This avoids a long blocking delay in the normal loop and helps keep the interface responsive.

6.2 Cell analysis

The firmware obtains the four cell readings, identifies the minimum and maximum readings, and calculates voltage imbalance as the difference between the strongest and weakest readings. It also compares the current imbalance with the previous measurement to label the trend as increasing, decreasing or stable. The threshold and SoC values in this prototype are demonstration logic and require calibration and validation before any real battery use.

6.3 Protection state machine

State	Intended meaning	Demonstration behaviour
NORMAL	Measurements are within configured demonstration limits	Relay is enabled and normal status is displayed
DEGRADED	A warning condition is present, such as high imbalance	System highlights the warning and recommends inspection/balancing
FAILSAFE	A serious detected fault requires protection	Relay output is switched OFF and fault status is displayed
SHUTDOWN	Manual or configured shutdown condition	System enters a more restrictive operating state until cleared by the implemented logic

6.4 LCD interface

The LCD rotates between pages showing cell voltages, system state/imbalance, SoC/risk score and communication/queue status. A fault display takes priority when a critical fault is active. The screenshots show these pages in operation.

6.5 Wi-Fi and Blynk telemetry

The dashboard displays the four cell voltages, estimated SoC, risk score, weakest and strongest cell indices, imbalance, system state, relay status, fault status, Wi-Fi signal strength and queue depth. Historical graphs are configured for the four cell-voltage datastreams.

6.6 Event queue note

The firmware reports queue depth and includes an offline-queue/reconnection demonstration. The captured screenshots show queue depth changing during communication conditions. The current evidence should not be interpreted as proof of durable, exactly-once cloud replay of every original event payload; that behaviour requires a dedicated reconnect test and payload-level verification.

7. Monitoring and Analytics

- Cell voltage: individual values are displayed in volts.
- Weakest/strongest cells: indices identify the minimum and maximum measured cell inputs.
- Voltage imbalance: strongest-cell voltage minus weakest-cell voltage, displayed in millivolts.
- State of charge: a software estimate intended for dashboard demonstration, not a chemistry-calibrated SoC measurement.
- Risk score: a heuristic demonstration score based on monitored conditions.
- Maintenance advice: a short message derived from the prototype's current condition.
- Communication indicators: Wi-Fi/Blynk status, RSSI and queue depth.
- Historical data: Blynk graphs provide a visual record of the four voltage datastreams.

8. Testing and Results

The following observations are visible in the shared simulation and dashboard screenshots. Values are representative snapshots from the simulation and can change when the potentiometers are adjusted.

Test	Observed evidence	Result
Normal monitoring	Four cell voltages around 3.6–3.8 V; NORMAL state; relay ON	Normal display path demonstrated
Imbalance warning	Imbalance around 156 mV with DEGRADED state in a captured run	Warning state displayed
Undervoltage injection	Cell 2 around 2.955 V; FAILSAFE; CELL_VOLTAGE fault; relay OFF	Protection response demonstrated
Recovery	Cell voltages restored; NORMAL state; fault ID 0; relay ON	Recovery path demonstrated in simulation
LCD pages	Cell voltages, SoC/risk/relay and Wi-Fi/queue pages visible	Local interface demonstrated
Blynk live view	Four voltage widgets and system indicators visible	Remote dashboard demonstrated
Historical graphs	Four voltage histories visible in Blynk	Graph widgets configured and receiving data
GitHub publication	Repository shows source file and README	Source repository created

The screenshots demonstrate expected simulation behaviour for the scenarios shown. They do not establish hardware measurement accuracy, real relay-contact feedback, battery safety certification, or full fault coverage.

9. Screenshots and Evidence

The following figures are included from the screenshots shared during development. Blynk credential areas have been redacted in the report copy where applicable.

Figure 1. Complete ESP32-based BMS circuit in Wokwi.

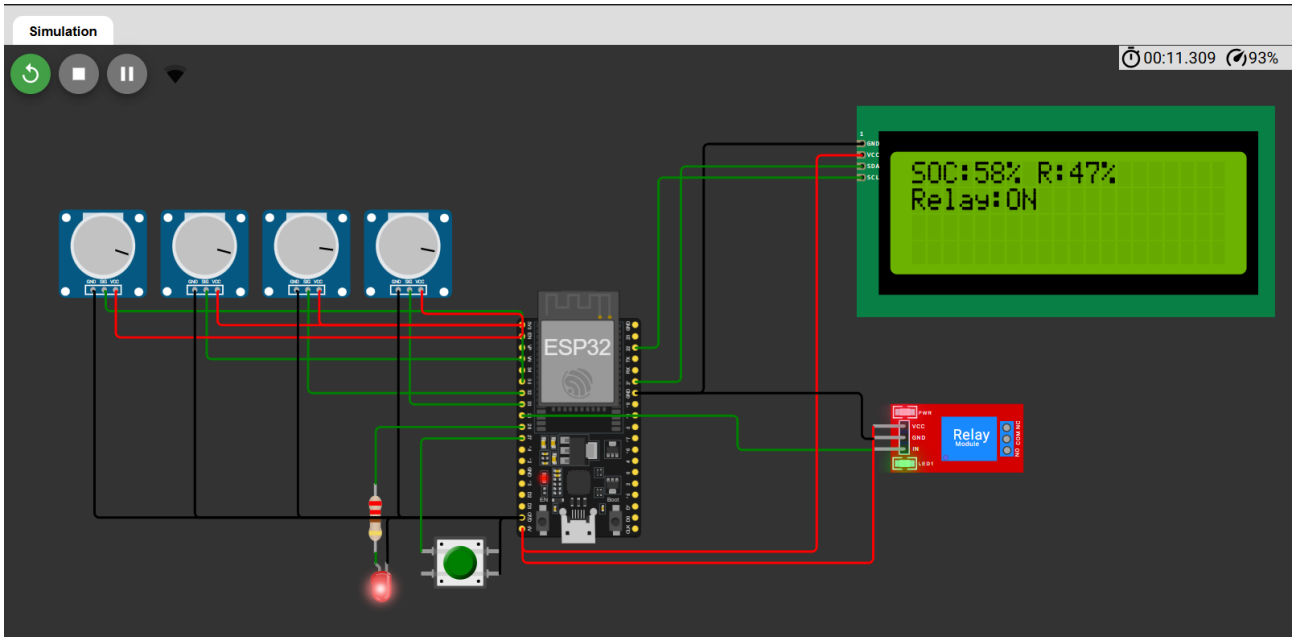
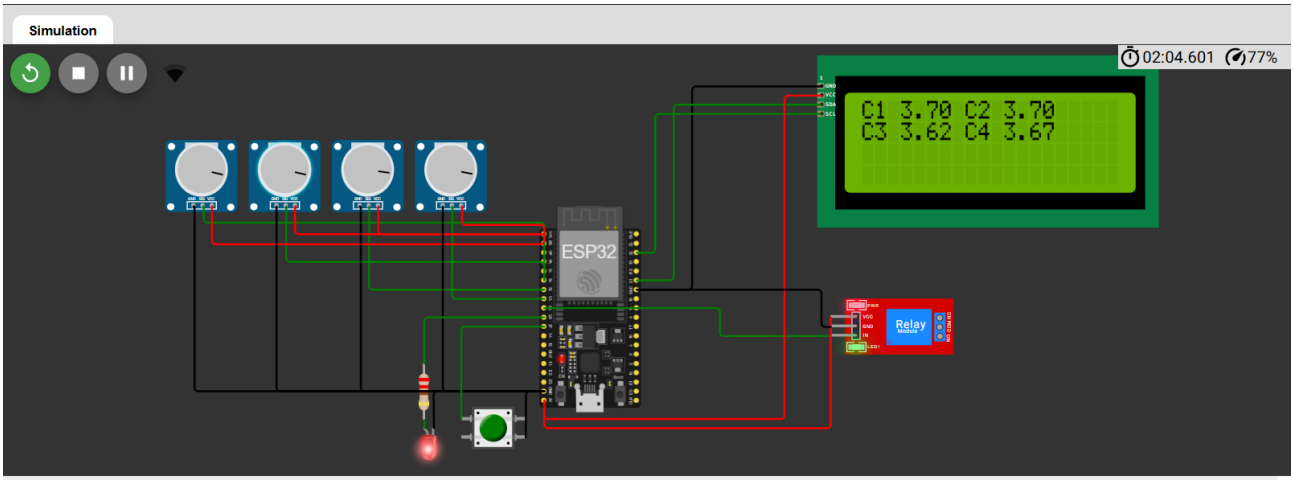
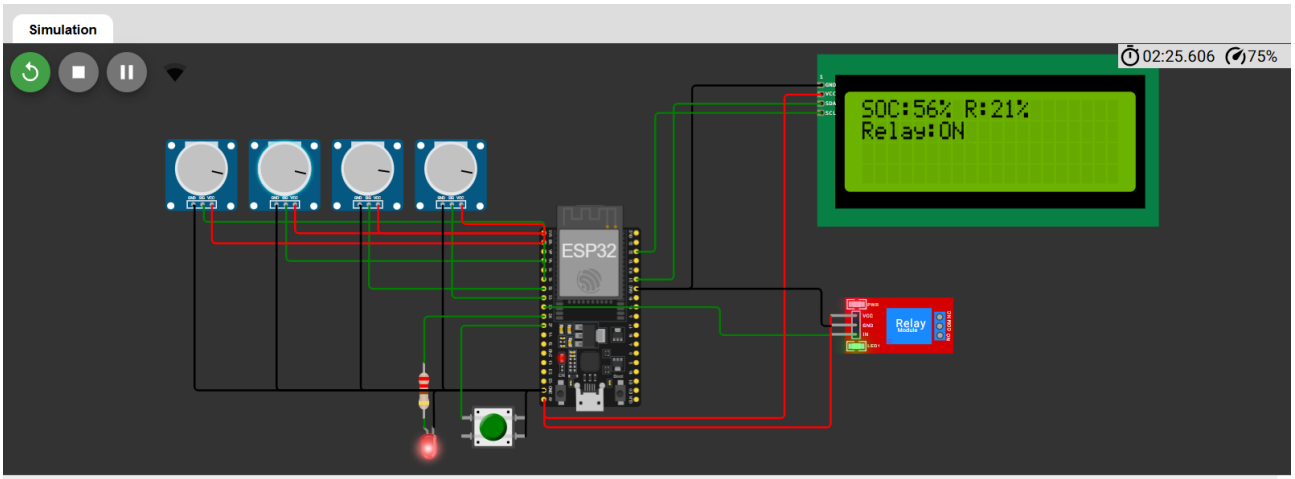


Figure 2. Normal battery monitoring: individual cell voltages, imbalance, system state and telemetry summary.



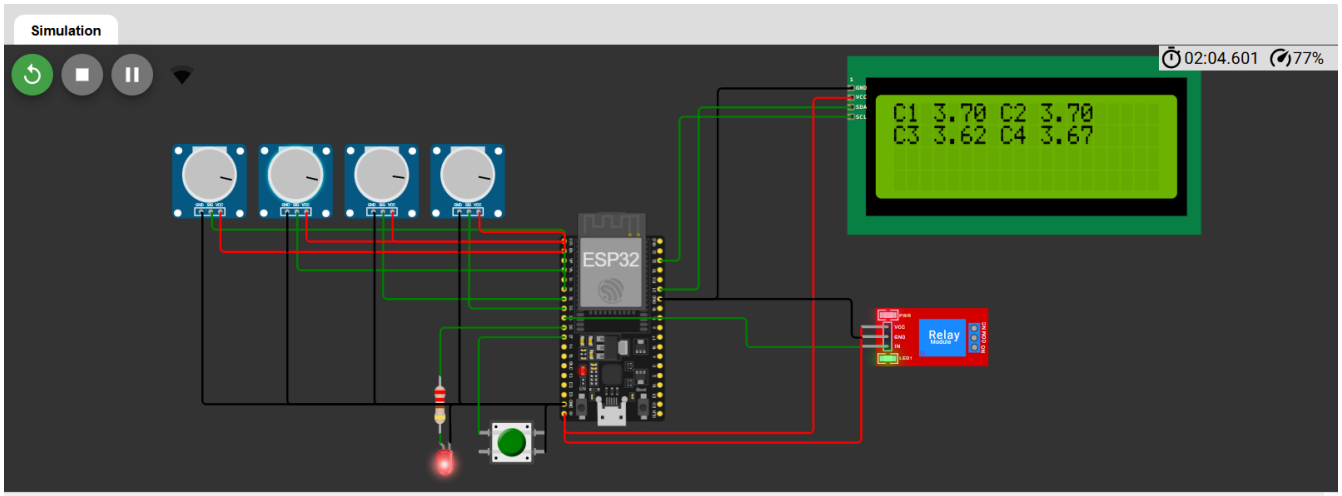

```
========== ESP32 INTELLIGENT BMS ==========
CELL1=3.703V
CELL2=3.703V
CELL3=3.625V
CELL4=3.670V
WEAK=C3 STRONG=C1
IMBALANCE=78mV LIMIT=100mV TREND=STABLE
SOC=56% STATE=NORMAL FAULT_ID=0 (NONE)
RELAY=ON WIFI=ONLINE RSSI=-63 QUEUE=0/8
RISK=21.0% UPTIME=121 s FAULT_COUNT=3
ADVICE=Battery operating OK
============================================
```

Figure 3. LCD page showing the estimated state of charge, risk score and relay status.




```
========== ESP32 INTELLIGENT BMS ==========
CELL1=3.703V
CELL2=3.703V
CELL3=3.625V
CELL4=3.670V
WEAK=C3 STRONG=C1
IMBALANCE=78mV LIMIT=100mV TREND=STABLE
SOC=56% STATE=NORMAL FAULT_ID=0 (NONE)
RELAY=ON WIFI=ONLINE RSSI=-67 QUEUE=0/8
RISK=21.0% UPTIME=142 s FAULT_COUNT=3
ADVICE=Battery operating OK
============================================
```

Figure 4. LCD page showing the voltage of all four monitored cells.




```
========== ESP32 INTELLIGENT BMS ==========
CELL1=3.703V
CELL2=3.703V
CELL3=3.625V
CELL4=3.670V
WEAK=C3 STRONG=C1
IMBALANCE=78mV LIMIT=100mV TREND=STABLE
SOC=56% STATE=NORMAL FAULT_ID=0 (NONE)
RELAY=ON WIFI=ONLINE RSSI=-63 QUEUE=0/8
RISK=21.0% UPTIME=121 s FAULT_COUNT=3
ADVICE=Battery operating OK
============================================
```

Figure 5. Communication/status page showing Wi-Fi state and event queue depth.

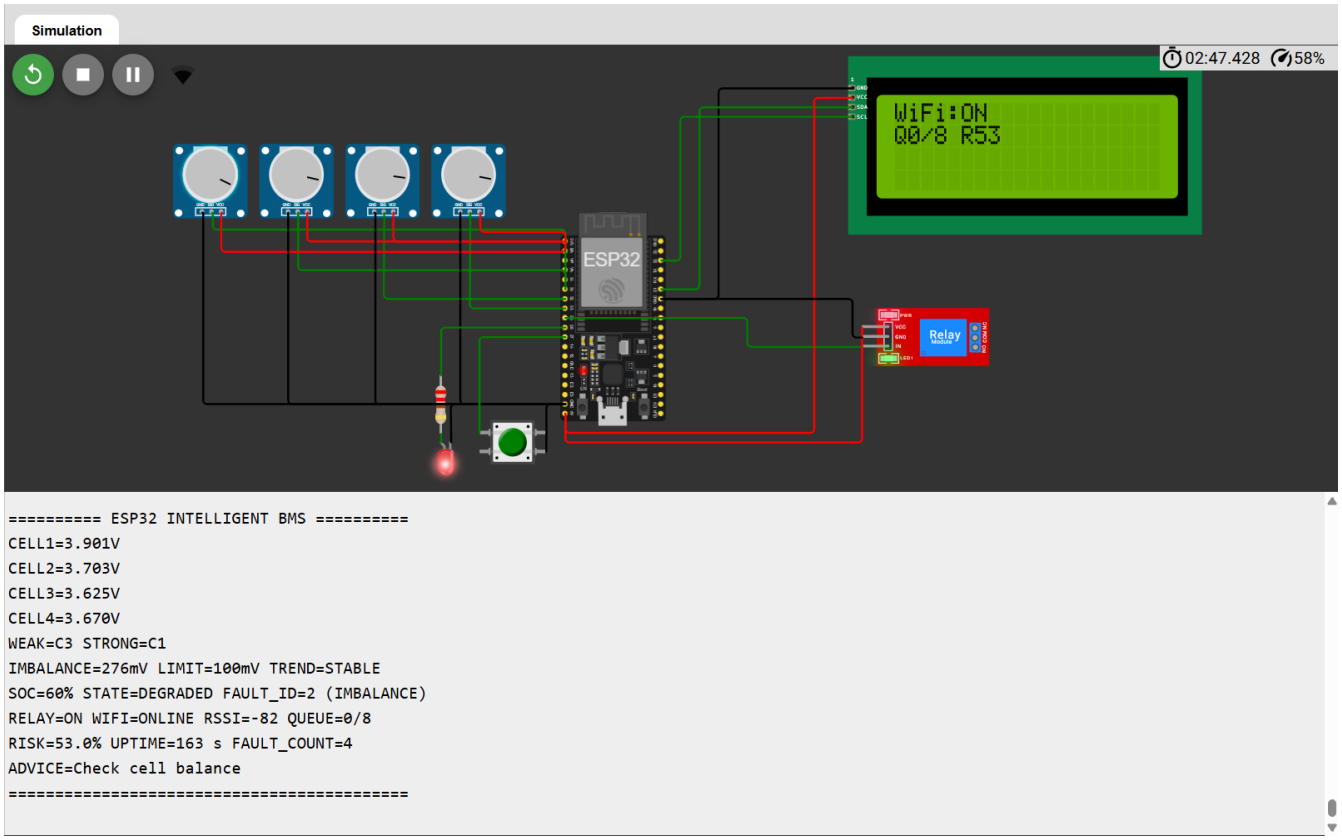
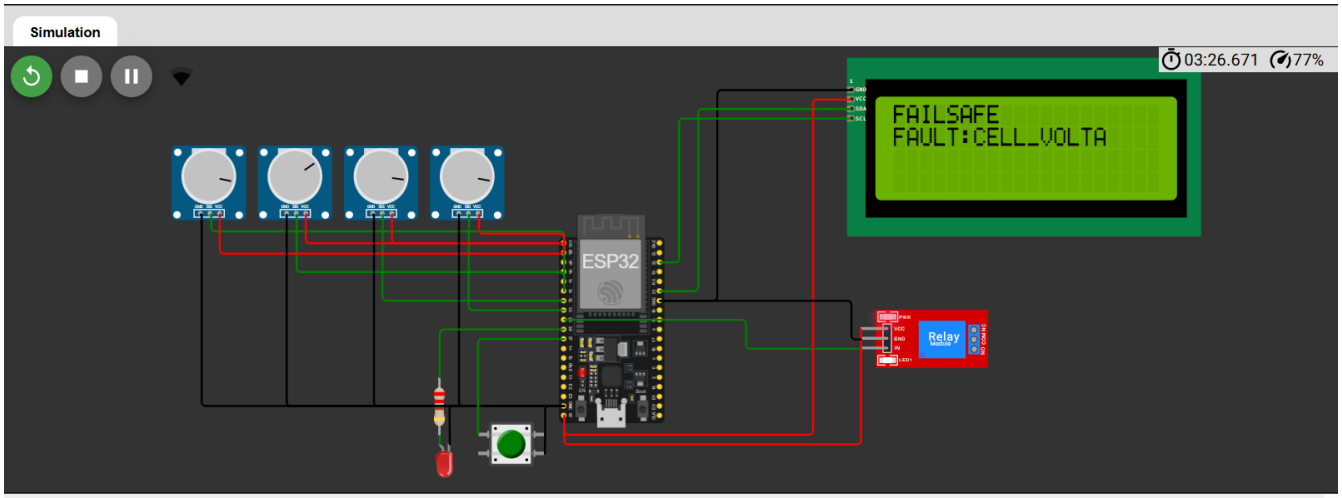


Figure 6. Undervoltage fault demonstration: cell voltage below 3.0 V, FAILSAFE state and relay OFF.




```
========== ESP32 INTELLIGENT BMS ==========
CELL1=3.684V
CELL2=2.955V
CELL3=3.625V
CELL4=3.670V
WEAK=C2 STRONG=C1
IMBALANCE=729mV LIMIT=100mV TREND=STABLE
SOC=40% STATE=FAILSAFE FAULT_ID=1 (CELL_VOLTAGE)
RELAY=OFF WIFI=ONLINE RSSI=-90 QUEUE=0/8
RISK=81.0% UPTIME=203 s FAULT_COUNT=8
ADVICE=Inspect fault now
============================================
```

Figure 7. Recovery demonstration: system returned to NORMAL after cell voltages were restored.

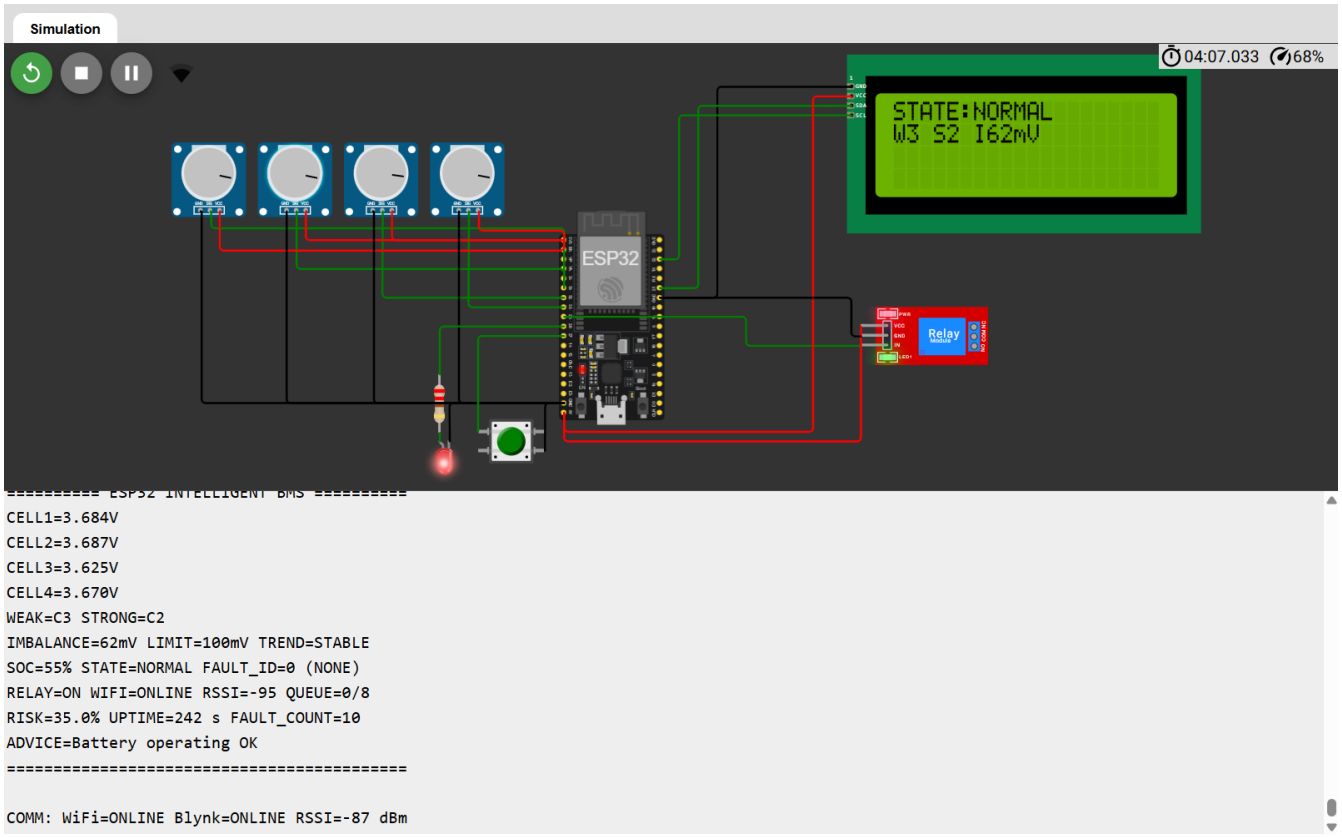


Figure 8. Blynk live dashboard with cell voltages, SoC, risk score and system indicators.

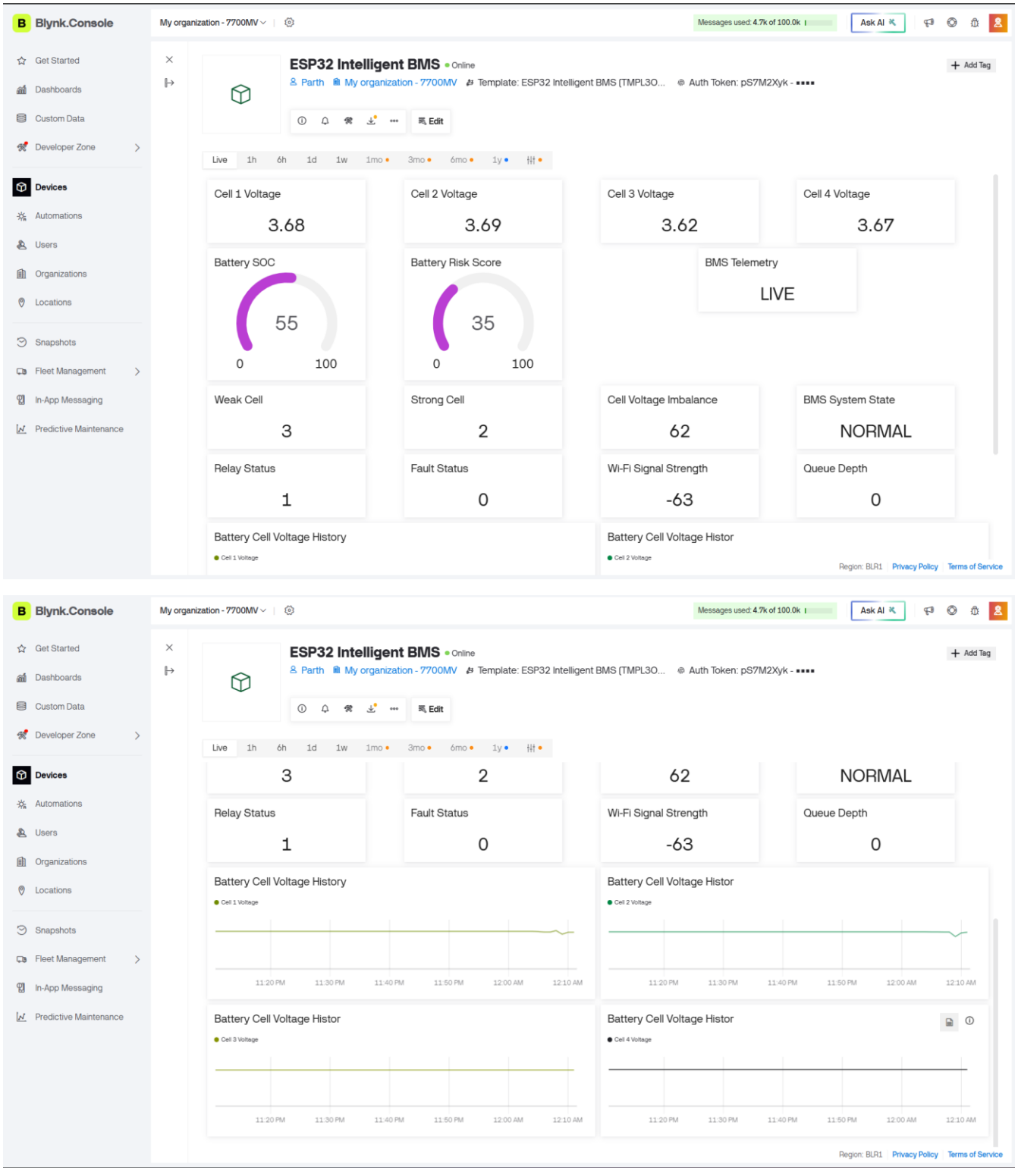
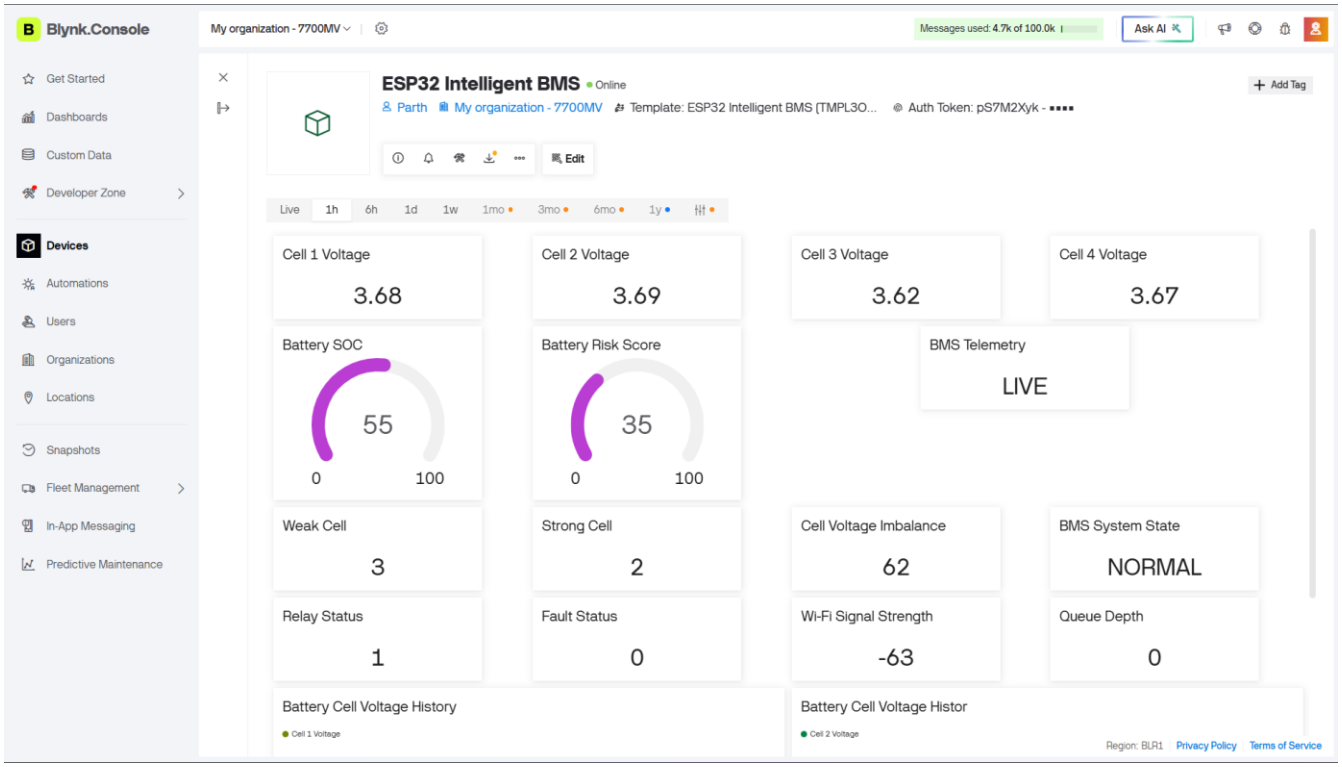


Figure 9. Blynk historical voltage graphs for all four cells & Blynk dashboard status indicators and historical cell-voltage trends.



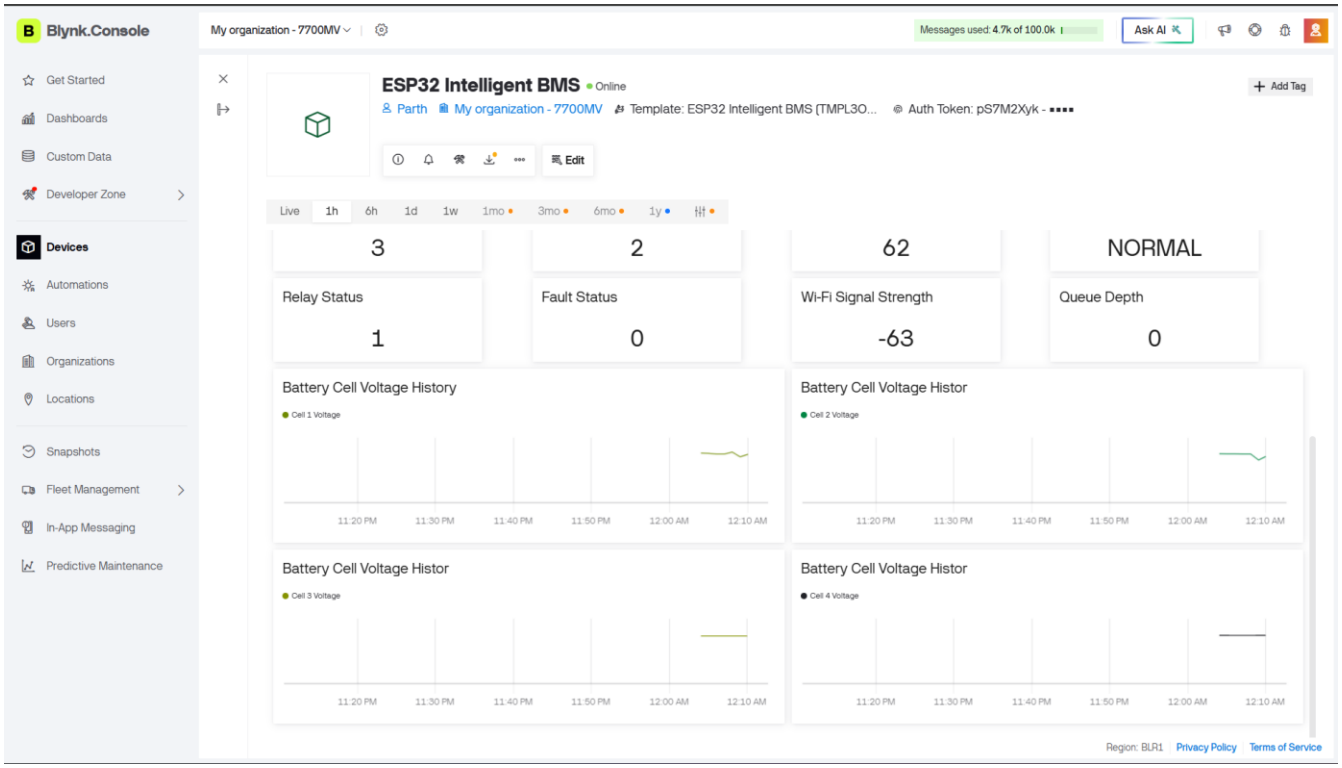
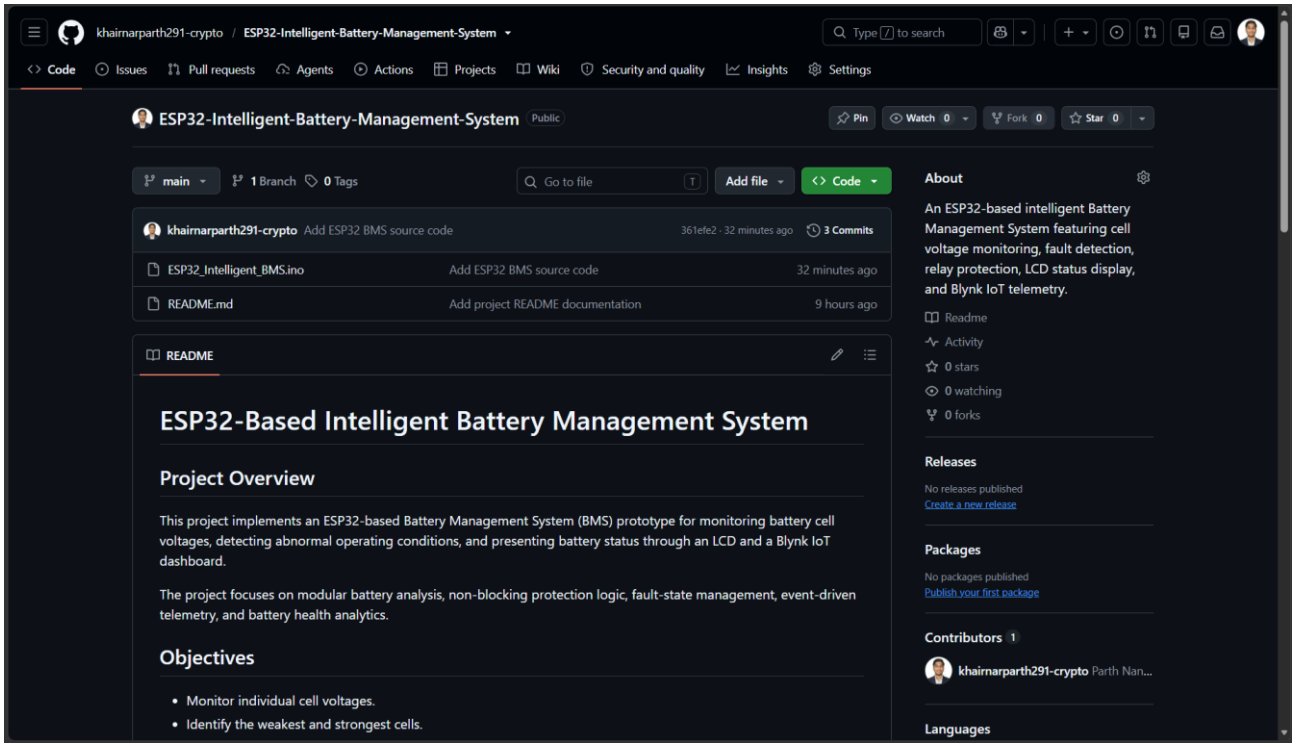


Figure 11. GitHub repository containing the project source file and README.



10. Limitations

- The prototype is tested in Wokwi using potentiometers as cell-voltage surrogates; it is not connected to a real battery pack.
- The displayed SoC and risk score are estimates/demo analytics and are not validated battery-management algorithms.
- Only four analog input channels are mapped in the current circuit; changing a cell-count constant alone does not create additional physical ADC inputs.
- No relay auxiliary-contact feedback is shown in the circuit, so actual contact mismatch cannot be confirmed from the current setup.
- A frozen ADC value within a normal range requires stronger detection logic than simple rail-value checks.
- Offline event queue behaviour and ordered replay should be verified with explicit network-loss and reconnect tests.
- The prototype does not implement cell balancing hardware, current measurement, temperature sensing, isolation monitoring or certified protection.

11. Future Scope

- Add calibrated voltage-sensing front ends and safe measurement isolation for a real multi-cell pack.
- Integrate current and temperature sensors and implement chemistry-appropriate SoC/SoH estimation.
- Add relay feedback contacts and validate relay state against the commanded state.
- Improve frozen-sensor diagnostics using time-series plausibility checks and redundant measurements.
- Implement and test bounded event storage with original event payloads, ordering, retry limits and explicit delivery acknowledgement.

- Persist fault history and analytics across restarts and add clearer dashboard severity indicators.
- Design and validate a modular input layer for additional cell counts instead of relying on a compile-time constant alone.
- Perform hardware-in-the-loop testing and independent safety review before any real battery application.

12. Conclusion

The ESP32 Intelligent Battery Management System prototype demonstrates a practical embedded workflow for monitoring multiple simulated cell voltages, identifying voltage imbalance, showing fault states, controlling a relay output, presenting local LCD pages and visualising telemetry through Blynk. The shared screenshots document normal operation, a simulated undervoltage event, recovery, dashboard indicators and historical graphs. The project provides a useful learning platform for embedded firmware, IoT telemetry and fault-state design. Further calibration, hardware integration and safety validation are required before it can be used with real battery cells.